

Question 1:

```
n = int(input())
lev = int(input())
p = [int(input()) for _ in range(n)]
b = [int(input()) for _ in range(n)]

a = sorted(zip(p, b))
ans = 0

for power, bonus in a:
    if power > lev:
        break
    lev += bonus
    ans += 1

print(ans)
```

Question 2:

```
def counter(n, k):
    num = 0
    if k == 1:
        return n
    else:
        for i in range(1, n + 1):
            for j in range(1, n + 1):
                if j % i == 0:
                    num += 1
    return num

def count(n, k):
    if k == 1:
        return n
    if k == 2:
        return counter(n, k)
    mid = k // 2
    x = count(n, k - mid)
    y = counter(n, mid)
    return x + y - 1

n = int(input())
```

```
k = int(input())
print(count(n, k))
```

Question 3:

```
def sub(arr, i, n, prev=0):
    if i == n:
        return 0
    a = sub(arr, i + 1, n, prev)
    b = 0
    if arr[i] > prev:
        b = 1 + sub(arr, i + 1, n, arr[i])
    return max(b, a)
```

```
n = int(input())
arr = []
for i in range(n):
    arr.append(int(input()))
print("Length of Bitwise subsequence will be", sub(arr, 0, len(arr)))
```

Question 4:

```
def Xor_sum(x, arr):
    xorSum = sum(arr)

    for i in range(1, x):
        s = 0
        for j in arr:
            s += i ^ j
        if s > xorSum:
            xorSum = s
    return xorSum
```

```
n = 4
x = 9
arr = [7, 4, 0, 3]
print(Xor_sum(x, arr))
```

Question 5:

```
def convertBase(m, base):  
    rem = m % base  
    m = m // base  
    while m >= base and (m % base == rem):  
        m = m // base  
    if m == rem:  
        return True  
    return False
```

```
m = int(input())  
base = 2  
while not convertBase(m, base):  
    base = base + 1  
print(base)
```

Question 6:

```
def convertBase(m, base):  
    rem = m % base  
    m = m // base  
    while m >= base and (m % base == rem):  
        m = m // base  
    if m == rem:  
        return True  
    return False
```

```
m = int(input())  
base = 2  
while not convertBase(m, base):  
    base = base + 1  
print(base)
```

Question 7:

```
def solution(nExercise, arr, energy):
    total = 0

    for i in range(nExercise):
        if arr[i] == energy:
            return 1

    for i in range(nExercise):
        total += arr[i]

        if arr[i] == energy:
            return 1

        if total >= energy:
            return i + 1

        if 2 * total >= energy:
            return 2 * (i + 1)

    return -1

if __name__ == "__main__":
    energy = int(input())
    nExercise = int(input())
    arr = list(map(int, input().split()))
    result = solution(nExercise, arr, energy)
    print(result)
```

Question 8:

```
def ans(arr, n, m, h):
    total = sum(arr)

    if total <= m * h:
        return 0
    else:
        sumh = 0
        totalhero = m * h
        for i in range(n - 1, 0, -1):
            sumh += arr[i]
            if sumh >= totalhero:
```

```
    return i
```

```
    return 0
```

```
if __name__ == "__main__":  
    n, m, h = map(int, input().split())  
    arr = list(map(int, input().split()))  
    print(ans(arr, n, m, h))
```